

Technical Report — Assignment 03: Self-Improving AI for Financial Question Answering

1. Thông tin cá nhân

Mục	Thông tin
Họ tên	Nguyễn Trường Sơn
Kaggle username	truongson1209
Kaggle team name	Son Nguyen
Hình thức nộp	Cá nhân (100% khối lượng công việc)
Video thuyết trình	https://drive.google.com/file/d/1Wkg4p0VsM9F1yfeT5RGDIOCynPAdNrgS/view?usp=sharing

2. Tóm tắt & Kết quả Kaggle

Hệ thống EvoAgent giải bài toán hỏi-đáp tài chính tiếng Việt (FinQA-style): đọc văn bản + bảng số liệu, sinh chương trình DSL phẳng (`subtract(a,b)`, `divide(#0,b)` ...), thực thi chương trình để ra giá trị số cuối cùng.

Kết quả chính:

Mốc	Dev accuracy	Kaggle Public LB
Baseline (EvoAgent thuần, iter_004, greedy)	0.5792 (dev-240)	0.59919
Hệ thống cuối (Giai đoạn 3)	0.7346 (full dev-584)	0.77935
Cải thiện	+15.5 pp	+18.0 pp

Submission cuối: **【Điền: tên submission trên Kaggle】** · Rank tại thời điểm nộp: **【Điền: rank/tổng số team】** (top LB lúc đó: 0.80971).

Ba nguồn cải thiện chính (chi tiết ở mục 5):

- Vá các lỗi hệ thống của baseline — request thất bại bị nuốt thành 0.0, chuỗi suy nghĩ chạy vô hạn, extraction gom rác, few-shot dạy sai cú pháp (~+6 pp).
- Self-consistency voting trên 16 candidates/câu (~+7 pp).
- Ensemble với retrieval few-shot per-question (~+2.4 pp).

3. Pipeline cuối cùng

Quy ước tên gọi theo đề bài: **Giai đoạn 2** = tái dựng vòng lặp tự cải thiện EvoAgent theo các khối TODO (Milestone 1 — chạy tiến hóa strategy, sinh `evolution_proof.json`, strategy tốt nhất `iter_004`); **Giai đoạn 3** = cuộc thi Kaggle mở, tự do kỹ thuật.

Hệ thống cuối của Giai đoạn 3 không xây từ con số 0: nó đứng trên bốn "tài sản" của Giai đoạn 2 (mục 3.2), rồi bọc quanh đó một chuỗi khối xử lý mới (mục 3.3).

3.1 Nhìn toàn cảnh: một câu hỏi đi từ đầu đến cuối

Ví dụ xuyên suốt (mẫu thật trong `train.json`, id `PM/2017/page_25.pdf-1`): ngữ cảnh gồm đoạn văn + bảng giá cổ phiếu PMI, trong đó có hai ô «31/12/2012: \$100.00» và «31/12/2013: \$108.50». Câu hỏi: *"Tỷ lệ tăng trưởng giá cổ phiếu của PMI từ năm 2012 đến 2013 là bao nhiêu?"*.

Model không được trả lời thẳng "8.5%" mà phải sinh **chương trình DSL phẳng**:

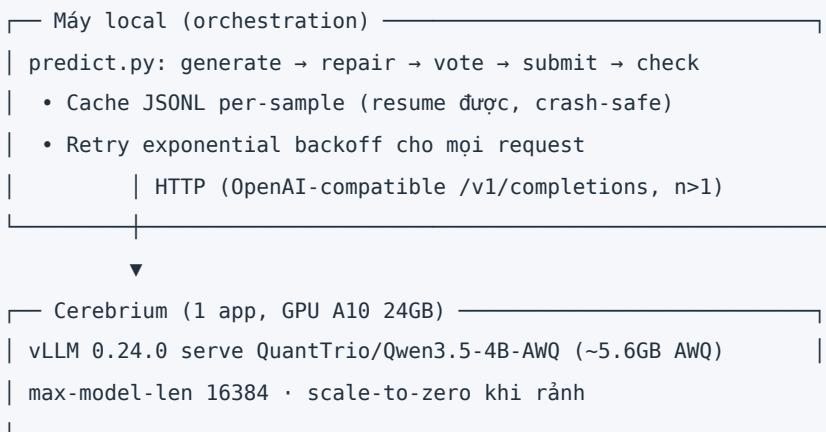
```
subtract(108.50, 100), divide(#0, 100)
```

(`#0` = kết quả bước trước). Hệ thống thực thi chương trình để ra giá trị nộp bài `0.085`. Mọi khối dưới đây đều phục vụ một mục tiêu duy nhất: **với mỗi câu hỏi, thu về một chương trình chạy được và đúng**.

Dòng chảy đầy đủ cho mỗi câu trong 494 câu test:

```
test.json (câu hỏi + văn bản + bảng)
▼
[1] Dựng prompt x2 strategy (s4fix / retr)      ← tài sản Giai đoạn 2: template + few-shot iter_004
▼
[2] Sinh 16 candidates = 2 strategy × (1 greedy + 7 sampled)  – vLLM trên Cerebrum
▼
[3] Extraction v2: bóc chương trình DSL từ raw output
▼
[4] Repair: sample nào bóc thất bại → re-ask guided-JSON 1 lần
▼
[5] Evaluator v2: THỰC THI từng chương trình → giá trị số'
▼
[6] Voting self-consistency: cluster theo giá trị → chọn cluster thắng
▼
[7] format_submission.py → predict.py check → submission.csv → Kaggle
```

Về hạ tầng, orchestration chạy local, inference chạy trên Cerebrum:



3.2 Tận dụng kết quả Giai đoạn 2

Giai đoạn 3 kế thừa bốn tài sản cụ thể từ Giai đoạn 2:

1. **Strategy vô địch của vòng tiến hóa (`iter_004`).** Toàn bộ prompt template + bộ few-shot của strategy tốt nhất mà vòng lặp propose → evaluate → reflect của EvoAgent tìm ra được dùng nguyên làm gốc cho strategy `s4fix` (chỉ sửa 2 chỗ, xem khối (b) mục 3.3). Đây là kết quả trực tiếp, đo được, của quá trình tự cải thiện.
2. **Few-shot CoT synthetic đã được verify bằng thực thi.** Các chuỗi reasoning trong few-shot do chính Qwen3.5-4B sinh ra trong vòng tiến hóa Giai đoạn 2, và chỉ được giữ lại khi chương trình đi kèm thực thi khớp đáp án gold — tức Giai đoạn 3 thừa hưởng dữ liệu dạy đã qua kiểm chứng, không phải viết tay.
3. **Artifacts làm "hồ sơ bệnh án" của baseline.** Raw output dev-240 và 494 raw output test của Giai đoạn 2 là bộ dữ liệu để điều tra lỗi per-question (chi tiết mục 5.1). Mỗi khối của Giai đoạn 3 sinh ra để trị đúng một bệnh trong hồ sơ đó:

Chẩn đoán từ artifacts Giai đoạn 2	Khối Giai đoạn 3 trị nó
Runaway CoT 21%, nhóm này đúng 0%	(c) decoding + (d) không mót rác + (e) repair
Request fail bị nuốt thành 0.0 (9.3% test)	(h) reliability layer
Extraction gom rác → crash evaluator	(d) extraction v2
Few-shot #4 dạy nested call không chạy được	(b) sửa few-shot + (f) flatten trong evaluator
Lỗi dấu/chiều thay đổi, số kiểu Việt	(b) 2 hint mới trong prompt

4. **Hạ tầng tái dùng.** App Cerebrum + vLLM giữ nguyên cấu hình serving; executor DSL và `TokenBudget` của Giai đoạn 2 được nâng cấp thành evaluator v2 (khối (f)); 2,986 cặp (câu hỏi, gold program) trong `data/train.json` trở thành kho tra cứu few-shot cho strategy retrieval.

3.3 Triển khai Giai đoạn 3: từng khối theo thứ tự dòng chảy

(a) Serving. Một app Cerebrum duy nhất (GPU A10 24GB) chạy vLLM serve `QuantTrio/Qwen3.5-4B-AWQ`, expose API OpenAI-compatible, scale-to-zero khi rảnh. Giữ nguyên từ Giai đoạn 2 — mọi cải thiện nằm ở phía client/pipeline, không đổi model hay serving.

(b) Dựng prompt — 2 strategies, cùng base template.

Strategy A (s4fix) = `iter_004` kế thừa từ Giai đoạn 2 với đúng 2 sửa đổi:

- few-shot #4 vốn dạy nested call `divide(subtract(500,400),400)` — chính evaluator của đề không chạy được — đổi thành dạng phẳng `subtract(500,400), divide(#0,400)` ;
- bổ sung 2 hint: quy tắc chiều thay đổi ("từ A đến B = B – A, kết quả có thể âm" — ví dụ câu PMI: `subtract(108.50, 100)` chứ không phải `subtract(100, 108.50)`) và quy tắc số kiểu Việt ("11.228 tỷ" = 11228, không phải 11.228).

Strategy B (retr) dùng **cùng template** nhưng thay few-shot cố định bằng **retrieval per-question**: TF-IDF char n-gram (2,4) trên 2,986 câu train, lấy top-8 gần nhất rồi chọn 3 shot đa dạng op-signature. Cơ chế gồm bốn bước:

1. Mỗi câu hỏi được cắt thành mọi cụm 2-4 ký tự liên tiếp (char n-gram — "doanh thu" sinh `do`, `oa`, ..., `doan`, `oanh`); so khớp ở mức ký tự nên chịu được biến thể chính tả tiếng Việt ("tỷ"/"tỉ" vẫn trùng phần lớn cụm) mà không cần model embedding.
2. TF-IDF đánh trọng số: cụm phổ biến ở mọi câu ("là bao", "nhiều") gần 0, cụm hiếm mang nội dung ("khẩu ha", "tỷ suất") trọng số cao — mỗi câu thành một vector và độ giống giữa hai câu = độ trùng hai vector (cosine).
3. Xếp hạng cả 2,986 câu train theo độ giống với câu đang hỏi, giữ 8 câu đầu làm sơ tuyển.
4. Prompt chỉ chứa được 3 shot, nhưng lấy thô top-3 thường được 3 bản sao cùng một khuôn tính — nên duyệt từ trên xuống và bỏ qua câu có **op-signature** (chuỗi tên phép tính của gold program, vd. `subtract(108.50,100), divide(#0,100)` → `subtract,divide`) trùng với shot đã chọn, để 3 shot cùng chủ đề nhưng trình diễn 3 khuôn giải khác nhau (nếu không gom đủ 3 signature khác nhau thì chấp nhận lấy trùng).

Minh họa thuật toán chọn ở bước 4 — duyệt top-8 từ trên xuống, câu nào có signature **chưa gặp** thì lấy, **gặp rồi** thì bỏ qua, đủ 3 thì dừng:

Hạng	Op-signature	Quyết định
1	<code>subtract,divide</code>	✓ lấy — shot 1 (khuôn mới)
2	<code>subtract,divide</code>	❌ bỏ (trùng shot 1)
3	<code>subtract,divide</code>	❌ bỏ (trùng)
4	<code>divide</code>	✓ lấy — shot 2 (khuôn mới)
5	<code>subtract,divide</code>	❌ bỏ (trùng)
6	<code>subtract,divide,multiply</code>	✓ lấy — shot 3 (khuôn mới) → dừng

Kết quả: 3 shot vẫn **cùng chủ đề** với câu đang hỏi (đều thuộc top-8 giống nhất) nhưng trình diễn **3 công thức giải khác nhau** — mỗi suất bài mẫu dạy được một điều mới, thay vì 3 bản sao của cùng một khuôn.

Lưu ý vai trò của từng loại dữ liệu: phép **so khớp chỉ dùng câu hỏi** (index TF-IDF xây trên 2,986 câu hỏi train — lúc suy luận, câu hỏi là thứ duy nhất ta có; câu test không có đáp án để so). **Gold program chỉ tham gia sau khi đã tìm xong** top-8, ở ba vai: bộ lọc đa dạng (op-signature ở bước 4), nội dung bài mẫu nhét vào prompt, và định vị cửa sổ ngữ cảnh ~500 ký tự quanh các con số của nó (bài mẫu chỉ chứa đúng khúc văn bản liên quan thay vì cả trang báo cáo). Câu train không có gold program bị loại khỏi index từ đầu.

Ví dụ với câu PMI, retrieval trả về các câu train cùng dạng "tỷ lệ tăng trưởng... từ năm X đến năm Y" kèm gold program phẳng của chúng — few-shot chính là chương trình chuẩn thực thi được, không

phải CoT tự sinh. Ensemble hai strategy chỉ khác nhau đúng một biến (nguồn few-shot) nên dễ quy kết nguyên nhân khi ablation.

(c) Sinh 16 candidates — decoding chống runaway. Baseline Giai đoạn 2 chạy greedy temp=0 không repetition penalty → 21% output rơi vào vòng lặp suy nghĩ vô hạn (model lặp mãi "kiểm tra lại..." và không bao giờ đóng `</think>`), nhóm này đúng **0%**.

Giai đoạn 3: mỗi strategy sinh 1 greedy (`repetition_penalty=1.05`, cap 3072 tokens) + 7 sampled (`temperature=0.6`, `top_p=0.95`, `top_k=20`, `repetition_penalty=1.05`, cap 2048) — 7 sampled gói trong **một request** `n=7` để chia sẻ prefill trên vLLM, chi phí gần bằng 1 request.

Tổng: $2 \times (1+7) = \mathbf{16 \text{ candidates/câu}}$; sampled tạo đa dạng cho voting, greedy làm mỏ neo ổn định.

Hai chú thích thuật ngữ:

- **Greedy vs sampled:** model sinh từng token một, mỗi bước có một bảng xác suất cho token kế tiếp. *Greedy* (temp=0) luôn bốc token xác suất cao nhất → deterministic, cùng prompt luôn ra cùng kết quả — nhưng khi đã rơi vào vòng lặp thì không tự thoát được (vì thế cần repetition penalty). *Sampled* gieo token theo phân phối (temperature) → mỗi lần sinh đi một ngã suy luận khác nhau, thăm dò được các lối giải mà greedy bỏ lỡ — điều kiện cần để voting có ý nghĩa (8 lần temp=0 sẽ ra 8 bản y hệt).
- **3 few-shot ≠ 8 lần sinh** — hai trục độc lập: few-shot là số bài mẫu ở **đầu vào** (nằm trong prompt, dạy trước khi làm); 1+7 là số lời giải lấy ở **đầu ra** (cả 8 lần sinh dùng chung đúng một prompt, chỉ khác ngã suy luận).

(d) Extraction v2 — bóc chương trình khỏi raw output. Output tử tế có dạng:

```
<think>...100 → 108.50, tăng 8.50, chia gốc 100...</think>
{"Program syntax": "subtract(108.50, 100), divide(#0, 100)", "Numerical result": 0.085}
```

Baseline gom mọi mảnh `op(...)` trong toàn bộ monologue thành một "chương trình" → 47/240 câu dev crash evaluator. Extraction v2 dùng ladder ưu tiên:

1. JSON sau `</think>` cuối cùng — như ví dụ trên;
2. JSON bị cắt cụt vì hết token (vd. `{"Program syntax": "subtract(108.50, 100), div`) — vẫn bóc được bằng regex theo key `"Program syntax"`;
3. quét từ dưới lên tìm dòng thuần-program.

Extraction chạy ở cấp **từng candidate** (16 lượt bóc/câu, mỗi raw output xử lý độc lập, mỗi candidate một dòng cache riêng); các bước (d)–(f) đều per-candidate, chỉ đến khối (g) voting 16 kết quả mới gộp lại thành một đáp án.

Tuyệt đối không mót fragment từ output runaway — nhóm đó trả rỗng và chuyển cho repair. Lý do: với voting, một phiếu **rỗng** vô hại (và còn được repair cứu), còn một chương trình rác-nhưng-chạy-được là một phiếu **độc** — nó thực thi ra một con số nghe hợp lý và kéo lệch cuộc bầu chọn. Thà phiếu trắng còn hơn phiếu giả.

Chương trình bóc ra còn phải qua 4 cửa validate cú pháp, trượt cửa nào loại cửa đó:

1. **Op hợp lệ** — từng bước phải gọi hàm có trong DSL; hàm model bịa ra (`percent(...)`, `sum(...)`) giữ lại chỉ để crash evaluator.
2. **#ref đúng thứ tự** — `#N` chỉ được trỏ về bước đã tính trước đó; tham chiếu "tương lai" (bước 2 gọi `#5`) không thể thực thi tuần tự.

3. **≤ 8 bước** — bài FinQA thật hiếm khi quá vài phép tính; "chương trình" dài hơn gần như chắc chắn là rác nổi từ output lặp, không phải lời giải.
4. **Dedupe bước lặp** — trường hợp cứu được thay vì loại: hai bước liên kế y hệt nhau (vết tích model lặp) thì gộp còn một.

(e) Repair pass — cơ hội thứ hai. Diện quét: mọi sample **không có chương trình thực thi được** — cụ thể là request fail, hoặc không bóc được chương trình (`program is None`), hoặc bóc được mà thực thi không ra giá trị (`value is None`). Năm nhóm nguyên nhân thực tế:

1. **Runaway — nhóm lớn nhất (~19.5% sample)**: model kẹt trong vòng lặp suy nghĩ, trần `max_new_tokens` chém đứt output khi vẫn còn trong `<think>` nên không tồn tại phần đáp án. Lưu ý: không phải "bài khó nên thiếu chỗ suy nghĩ" — model lặp vô hạn, cho thêm token cũng không dừng; trần chỉ là cái cắt cụt.
2. Đóng `</think>` tử tế nhưng **JSON đáp án bị cắt cụt quá sớm** — cũng chạm trần token nhưng ở pha viết đáp án; nếu chuỗi chương trình đã kịp viết xong thì nấc 2 của ladder cứu được bằng regex, cắt sớm hơn nữa mới rơi xuống repair.
3. Output hoàn chỉnh nhưng **chương trình trượt validate** (bịa hàm ngoài DSL, `#ref` sai thứ tự, >8 bước) hoặc trả lời văn xuôi/đáp án thẳng không kèm chương trình.
4. Chương trình **hợp lệ cú pháp nhưng thực thi lỗi** — table op không khớp được hàng nào (kể cả fuzzy match), chia cho 0.
5. **Request fail hãn** — hiếm sau reliability layer, quét cho đủ.

Cách vá: đúng 1 lần re-ask — gửi lại 1200 ký tự cuối của bản nháp lỗi, ép trả lời bằng **guided-JSON decoding** theo schema `{"Program syntax": str}`, cap 256 tokens. Thiết kế nhằm thẳng nhóm 1: trong văn bản runaway, model thường **đã tính ra đáp án** — nó chỉ không chịu dừng để chốt; repair tước quyền suy nghĩ tiếp (256 token không đủ chỗ lan man), chỉ cho phép chốt chương trình từ những gì đã nháp. Vì vậy vá thành công 660/987 sample hổng chỉ trong 6.5 phút — không giải lại bài, chỉ "bắt nộ phần đã làm".

(f) Evaluator v2 — thực thi chương trình ra giá trị số. Nâng cấp executor của Giai đoạn 2 với ba sửa đổi:

1. Tự động **flatten nested call** — nếu model vẫn viết `divide(subtract(108.50,100),100)` thì chuyển thành `subtract(108.50,100), divide(#0,100)` thay vì crash.
2. **Fuzzy row match** cho table ops (bỏ dấu, hoa-thường, hậu tố đơn vị, Jaccard ≥ 0.6) — ví dụ chương trình gọi hàng "P/E" nay khớp được header thật "P/E (x)".
3. **Chuẩn hóa số âm kế toán** — ô `"$ -61.1 ( 61.1 )"` đọc thành `-61.1` (bug gốc khiến `table_max` chọn sai dấu; chính doc-test của `evaluator.py` để phát cũng sai vì bug này).

(g) Voting self-consistency — chọn giá trị cuối. Cả 16 candidates được thực thi hết và **vote theo GIÁ TRỊ đã execute**, không vote theo text (hai chương trình viết khác nhau nhưng cùng ra 0.085 vẫn là một phiếu chung). Các giá trị được cluster single-link với tolerance `max(1e-4, 0.5%·|v|)` — 0.085 và 0.08498 vào chung cluster.

Tolerance cần thiết vì số thực hiếm khi bằng nhau tuyệt đối (làm tròn khác nhau ở bước trung gian); so bằng `==` sẽ xé vote thành 16 phe lẻ. Nó là **dung sai kép** — lấy vẻ lớn hơn, mỗi vẻ cứu một thái cực:

- Sàn tuyệt đối `1e-4` cứu các **đáp án nhỏ** quanh 0: với đáp án 0.001, dung sai tương đối 0.5% chỉ là $5 \cdot 10^{-6}$ — khắt khe phi lý so với sai số làm tròn.

- Về tương đối $0.5\% \cdot |v|$ cứu các **đáp án lớn**: hai lời giải ra 11,228 và 11,230 rõ ràng cùng đáp án, nhưng dung sai $1e-4$ sẽ tách chúng; 0.5% của 11,228 ≈ 56 mới đúng thang đo.

Bản chất: dung sai **tự co giãn theo thang đo của đáp án** — đáp án dạng tỷ lệ thì khắt khe cỡ phần vạn, đáp án dạng nghìn tỷ thì nói ra vài chục.

Ví dụ câu PMI giả định: 11 candidates ra ≈ 0.085 (gồm cả 2 greedy), 2 ra 8.5 (quên chia 100), 1 ra 0.185, 2 hỏng không thực thi được $\rightarrow$ cluster 0.085 thắng áp đảo.

Trọng số mỗi phiếu: 1.0, +0.5 nếu là greedy, +0.25 nếu giá trị execute khớp "Numerical result" model tự khai; giá trị đại diện = của greedy member trong cluster thắng (nếu có). Chỉ chấp nhận khi đủ đồng thuận: $\text{confidence} \geq 0.5$, ≥ 3 thành viên, $\text{margin} \geq 0.15$ so với cluster nhì.

Ba ngưỡng đồng thuận (hàng số tự đặt trong `VoteConfig` của `voting.py`, không học từ dữ liệu) đo ba khía cạnh khác nhau của một chiến thắng đáng tin:

Ngưỡng	Công thức	Ý nghĩa
$\text{confidence} \geq 0.5$	$\text{trọng số cluster thắng} \div \text{tổng trọng số mọi phiếu}$	thắng phải quá bán — chiếm 30% trong khi 70% còn lại rải rác thì chưa tin được
$\text{members} \geq 3$	số phiếu trong cluster thắng	ít nhất 3 lời giải độc lập cùng ra một số — 1-2 phiếu trùng nhau có thể chỉ là ăn may
$\text{margin} \geq 0.15$	$(\text{cluster nhất} - \text{cluster nhì}) \div \text{tổng}$	thắng phải cách biệt — nhất 40% nhì 38% là hai đáp án đang giằng co

Lưu ý ngữ nghĩa: trượt ngưỡng **không làm mất đáp án** — vote vẫn trả về giá trị cluster thắng; ba ngưỡng chỉ gắn cờ `needs_judge` (kèm lý do `low_confidence` / `few_members` / `low_margin`) để quyết định câu nào đáng gửi trọng tài trong thử nghiệm A5. Ở hệ thống cuối A4 (không judge), cờ này không kích hoạt gì — mọi câu đều dùng đáp án cluster thắng.

(h) Reliability layer — xuyên suốt mọi request. Điều tra Giai đoạn 2 phát hiện **46/494 câu test (9.3%) có raw output RỖNG** — request fail lúc pod restart bị nuốt im lặng thành 0.0. Giai đoạn 3 xử lý bằng hai lớp:

- Retry exponential backoff + jitter (phân loại transient 429/5xx/timeout — thử lại, vs permanent 4xx — dừng).
- Scheduler `as_completed` ghi ngay từng kết quả vào cache JSONL, mỗi dòng một sample với key `(qid, strategy, sample_index)` + status.

Đứt mạng hay tắt terminal giữa chừng $\rightarrow$ chạy lại **đúng lệnh cũ**, cache tự resume phần thiếu, không tốn lại GPU. Kết quả: **0/4,672 request thất bại** trên dev, 0 câu bị 0.0 oan trên test.

(i) Post-processing & submission gate. Giá trị cuối của 494 câu qua `format_submission.py` (dùng nguyên trạng của đề — giữ đúng thứ tự row, điền 0.0 câu thiếu); sau đó gate `predict.py check` phải PASS trước khi upload:

- đúng 3 cột `id, Usage, predicted_value`, đủ 494 id đúng thứ tự;
- số câu $0.0 \leq 10$ (baseline có tới 102!);
- không scientific notation/−0.0;
- phân bố giá trị tương đồng dev.

Chỉ upload Kaggle một lần với file đã qua gate.

3.4 Những gì KHÔNG dùng trong hệ thống cuối

- **Fine-tuning**: không dùng (giữ nguyên serving config; xem mục 7).
- **LLM-as-judge**: đã thử nghiệm đầy đủ với DeepSeek-R1-Distill-Qwen-7B nhưng **loại khỏi pipeline cuối** vì ablation trên dev cho kết quả âm (mục 5.3).

4. Dữ liệu ngoài & dữ liệu synthetic

- **Dữ liệu ngoài: KHÔNG sử dụng.** Toàn bộ few-shot (cả cố định lẫn retrieval) lấy từ `data/train.json` được cung cấp; không crawl, không dataset bổ sung, không tra cứu/tái tạo nhãn test dưới bất kỳ hình thức nào.
- **Dữ liệu synthetic**: các chuỗi reasoning CoT trong few-shot của strategy s4fix do chính pipeline EvoAgent (Qwen3.5-4B) sinh ra ở Giai đoạn 2 (vòng tiến hóa) và được verify bằng cách thực thi chương trình (chỉ giữ khi kết quả khớp gold). Không dùng model ngoài để sinh nhãn hay pseudo-label.
- **Model sử dụng**: `QuantTrio/Qwen3.5-4B-AWQ` (4B, model chính — toàn bộ generation/repair) và `deepseek-ai/DeepSeek-R1-Distill-Qwen-7B` (7B, self-host thử nghiệm judge — không có trong pipeline cuối). Cả hai $\leq 9B$, tuân thủ luật cuộc thi; không dùng API model đóng nào.

5. Thí nghiệm, Ablation & Lựa chọn hệ thống cuối

5.1 Điều tra lỗi baseline (động lực thiết kế)

Phân tích per-question trên artifacts của baseline (dev-240 + 494 raw output test) xác định điểm rơi:

Lỗi hệ thống	Bằng chứng	Tỷ trọng
Runaway CoT (không đóng <code></think></code>)	51/240 dev → 0% đúng	~50% tổng lỗi dev
Request fail bị nuốt thành 0.0	46/494 test raw output rỗng	9.3% test
Extraction gom rác → crash evaluator	47/240 dev	~46% lỗi dev
Few-shot #4 dạy nested call	28/494 test program nested → 0.0	5.7% test
Lỗi dấu/chiều thay đổi	7/240 dev có pred = -gold	~3%

Phát hiện then chốt: trong các output đóng `</think>` tử tế, **100% chứa khối JSON parse được** → chỉ cần diệt runaway + extraction đúng chỗ là giải quyết phần lớn.

5.2 Bảng ablation chính (full dev-584, scorer abs $\leq 1e-4$)

#	Cấu hình (cộng dồn)	Dev acc	Δ
—	Baseline iter_004 (dev-240, tham chiếu)	0.5792	—

#	Cấu hình (cộng dồn)	Dev acc	Δ
A1	Greedy + toàn bộ fixes (decoding, extraction v2, evaluator v2, s4fix, repair)	0.6404	+6.1 pp
A3	+ Self-consistency K=8, vote theo giá trị	0.7106	+7.0 pp
A4	+ Ensemble strategy retrieval (16 candidates) — HỆ THỐNG CUỐI	0.7346	+2.4 pp
A5	+ LLM-judge R1-Distill-7B cho câu bất đồng thuận	0.7295	-0.5 pp (loại)

Accuracy theo loại câu hỏi (A4 vs baseline): `table_op` 0.33 → **0.71**; `division` 0.68 → 0.79; `addition` 0.39 → 0.65 (vẫn yếu nhất); `subtraction` 0.62 → 0.73; `multiplication` 0.60 → 0.80.

5.3 Ablation âm: LLM-as-judge (bằng chứng phương pháp luận)

Thử nghiệm: các câu vote không đạt ngưỡng đồng thuận (~20-35%) được đưa cho DeepSeek-R1-Distill-Qwen-7B (self-host trên cùng app Cerebrium, swap model tuần tự) phán xử. Thiết kế tôn trọng đặc thù reasoning model:

- **Không chặn thẻ `<think>`** (không ép JSON từ token đầu — làm vậy giảm mạnh chất lượng suy luận).
- Verdict bóc bằng regex sau `</think>` (`SELECTION/PROGRAM/FINAL_VALUE`).
- Chỉ tin chương trình judge viết lại **sau khi thực thi local thành công**.

Kết quả trên dev: judge override 33 câu, làm **giảm** 3 câu rỗng (0.7346 → 0.7295) — model 7B không đủ tin cậy để thắng majority vote của 16 candidates trên miền tài chính tiếng Việt. **Quyết định: loại judge khỏi hệ thống cuối, giữ A4.**

Toàn bộ chuỗi suy nghĩ của judge được lưu tại `runs/phase3/judge_cache.jsonl` làm bằng chứng.

5.4 Phương pháp validate

Toàn bộ quyết định dựa trên **full dev 584** (baseline chỉ dùng slice 240); test không có nhãn nên tuyệt đối không tune trên test; chỉ submit Kaggle 2 lần (baseline + final), không probe leaderboard.

6. Tái lập (Reproduction)

6.1 Môi trường & dependencies

- Python 3.13 (Anaconda env), các gói chính: `openai`, `transformers` (tokenizer-only), `datasets`, `scikit-learn`, `pydantic`, `python-dotenv`, `numpy`, `tqdm` (xem `requirements.txt`).
- Server: Cerebrium 1 app A10 — `cerebrium.toml` (vLLM 0.24.0 cài qua pip, `VLLM_USE_FLASHINFER_SAMPLER=0`, `--max-model-len 16384 --gpu-memory-utilization 0.90`).
- `.env`: `CEREBRIUM_BASE_URL`, `CEREBRIUM_API_KEY`, `CEREBRIUM_MODEL=QuantTrio/Qwen3.5-4B-AwQ`, `HF_TOKEN`.

6.2 Lệnh tái lập đầy đủ

```
# Dev (validate):
python predict.py generate --split validation --strategies s4fix --groups greedy,sampled --k
7 --cache runs\phase3\dev_cache.jsonl --concurrency 24
python predict.py repair --split validation --cache runs\phase3\dev_cache.jsonl
python predict.py generate --split validation --strategies retr --groups greedy,sampled --k 7
--cache runs\phase3\dev_cache.jsonl --concurrency 24
python predict.py repair --split validation --cache runs\phase3\dev_cache.jsonl
python predict.py score --ablation A1 ; python predict.py score --ablation A3 ; python
predict.py score --ablation A4

# Test + submission:
python predict.py generate --split test --strategies s4fix,retr --groups greedy,sampled --k 7
--cache runs\phase3\test_cache.jsonl --concurrency 24
python predict.py repair --split test --cache runs\phase3\test_cache.jsonl
python predict.py submit --ablation A4 --cache runs\phase3\test_cache.jsonl --out
runs\phase3\test_predictions.json
python format_submission.py --predictions runs\phase3\test_predictions.json --output-file
submission.csv
python predict.py check --submission submission.csv
```

Mọi lệnh idempotent (cache JSONL tự resume); log tự ghi vào `runs/phase3/run.log` , tham số snapshot vào `phase3_config.json` .

6.3 Cấu hình model & sampling

Tham số	Greedy	Sampled (n=7)	Repair	Judge (thử nghiệm)
temperature	0.0	0.6	0.0	0.6
top_p / top_k	1.0 / —	0.95 / 20	1.0 / —	0.95 / —
repetition_penalty	1.05	1.05	1.0	1.0
max_new_tokens	3072	2048	256 (guided JSON)	4096

Vote: tolerance `max(1e-4, 0.5%)` , ngưỡng chấp nhận confidence 0.5 / members 3 / margin 0.15.

6.4 Compute, chi phí & commit

Hạng mục	Input tokens	Output tokens
Generation (dev + test, Qwen)	55,941,840	17,239,308
Repair (Qwen, guided)	8,792,622	170,416
Judge thử nghiệm (R1-7B)	118,240	41,844
Tổng	~64.9M	~17.5M

- GPU: ~14-15 giờ A10 (Cerebrum, scale-to-zero khi rảnh). Chi phí thực tế: 【Điền: tổng \$ từ Cerebrum Dashboard → Billing, ước tính ~\$15-20】 .
- Không phát sinh chi phí API ngoài (judge tự host).
- Commit hash: 【Điền: hash sau commit cuối — tại thời điểm draft là `6df8af0657b1c629013954bb80a133d2a0a3318f` , cần commit nốt các thay đổi pipeline Giai đoạn 3 rồi cập nhật】 .

7. Hạn chế & Bài học

Hạn chế:

1. **Runaway CoT chưa trị tận gốc** — repetition penalty 1.05 giảm nhưng vẫn ~19.5% sample runaway; hệ thống bù bằng vote + repair thay vì sửa tận gốc (cần fine-tune hoặc model mới hơn).
2. **addition vẫn yếu nhất (0.65)** — thực chất là nhóm câu đa bước phức tạp bị classifier gán nhãn addition; cần phân loại câu hỏi tốt hơn để nhắm few-shot chính xác.
3. **Dev-LB lệch +4.5pp** (0.735 vs 0.779) — scorer nội bộ $\text{abs} \leq 1\text{e-}4$ khắt khe hơn metric Kaggle; may mắn lệch theo hướng có lợi nhưng làm giảm độ chính xác của dự báo dev.
4. **Judge 7B thất bại** — với budget model nhỏ, verifier không thẳng được self-consistency; muốn judge hiệu quả cần model lớn hơn đáng kể hoặc verifier được huấn luyện chuyên biệt.
5. Chưa thử fine-tuning (ràng buộc giữ nguyên serving + thời gian) — hướng tiềm năng nhất còn lại.

Bài học lớn nhất: phần lớn điểm cải thiện (+11 trong +18 pp) đến từ **kỹ nghệ hệ thống** — điều tra per-question để tìm đúng chỗ rơi điểm (request bị nuốt, extraction gom rác, few-shot dạy sai, decoding sai chế độ) — chứ không phải từ model thông minh hơn.

"Đo trước, sửa đúng chỗ, mỗi thay đổi một ablation" hiệu quả hơn nhiều so với đổi kiến trúc theo cảm tính; và một ablation âm được đo nghiêm túc (judge) cũng giá trị không kém ablation dương.

Phụ lục A — Bảng chứng đính kèm gói ThinkFlic

File	Nguồn
evidence/evolution_proof.json	./evolution_proof.json (Giai đoạn 2, nguyên trạng)
evidence/failure_mode_report.pdf	./runs/exp_self/failure_mode_report.pdf
evidence/learning_curve.pdf	./runs/exp_self/learning_curve.pdf
evidence/strategy_diversity.pdf	./runs/exp_self/strategy_diversity.pdf
(bổ sung Phase 3) runs/phase3/ablation_report.json , token_usage.json , run.log	sinh tự động bởi pipeline